Circuit Board Inspection

The Solar Car Team at UVA operates one of the largest printed circuit board (PCB) manufacturing plants in the 22904 zip code. Every board that rolls off the production line is scanned by an automated optical inspection system. The scanner converts each board into a two-dimensional character grid, where each character represents what the sensor detected at that grid position: for example, 'X' for a solder joint, 'O' for a copper trace, 'Y' for an LED, and'.' for bare substrate.

UVA Solar Car's engineering team has identified a specific rectangular grid of characters that indicates a known manufacturing defect called a misprint signature. Whenever this exact 2D pattern of characters appears anywhere on a board scan, the board is considered faulty and must be pulled from the production line before it ships to customers.

Because boards can be very large and the defect pattern may appear many times in a single scan, a naïve search is far too slow. Your job is to write a program that efficiently locates every occurrence of the defect pattern in the board scan and reports the total count and the top-left position of each match.

Input

The first line of input contains two integers R and C which represent the number of rows and columns of the board scan, where $1 \leq R, C \leq 5{,}500$.

The next R lines each contain exactly C characters with no spaces, representing the board scan.

The following line contains two integers r and c which represent the dimensions of the defect pattern, where $1 \leq r \leq R$ and $1 \leq c \leq C$.

The next r lines each contain exactly c characters representing the defect pattern.

Output

Print a single integer K which represents the number of times the defect pattern appears in the board scan.

Then print K lines, each containing two space-separated integers row col giving the 0-indexed top-left corner of each match, sorted first by column then by row.

If the defect pattern does not appear in the scan, print 0 and nothing else.

Sample Input

```
5 9
....XOX..
...XOXOX.
..XOXOXOX
...XOXOX.
....XOX..
3 3
XOX
OXO
XOX
```

Sample Output

```
4
1 3
0 4
2 4
1 5
```

Sample Input

```
3 5
XOXOX
OXOXO
XOXOX
2 3
XOO
OXX
```

Sample Output

```
0
```